Thakaa at AISA-ArabicFC: Convention-Aligned Prompting and the Limits of Argument Exact Match in Arabic Function Calling

Hassan Alqaeri **Meshal Alamr** **Abdullah Aldahlawi**
Thakaa, Advanced Company for Information Technology
Riyadh, Saudi Arabia
{h.alqaeril, m.alamr, aldahlawi}@thakaait.net

Abstract

Argument exact-match (ArgEM) dominates the AISA-ArabicFC score. Under one prompt, a base `gemma-4-12B-it` reaches 0.9853 FnAcc but only 0.6940 ArgEM on dev, suggesting that argument convention rather than tool selection is the binding constraint. Fine-tuning on the provided train split raises ArgEM to 0.8880 while FnAcc increases only from 0.9853 to 0.9945, yet the model still fails 8.4% of the 10,500 positive training rows. Across rules, retrieval, prompting, selection, reranking, and fusion, dev-selected gains did not consistently transfer beyond development; smaller gains from other tested interventions remain unresolved. Explicit convention cards add 7.5 ArgEM points on a dev-selected 228-row subset for an unconfigured model, but give no measurable gain to the already configured pipeline. A scorer-equivalent gold answer occurs in a 20-sample pool for 34/56 (60.7%) rows missed by the raw fine-tune, but our selectors cannot exploit it. Our official prompt-only `gemma-4-31B-it` submission placed 6th of 17 on Track A, 5th of 17 on Track B, and 4th of 21 on Track C. The 12B fine-tune used for analysis was developed first but inadvertently omitted from the blind-test submission, so it has no official test score.

1 Introduction

Function calling asks a model to map a natural-language request onto a structured API call: choose the correct tool from a catalog, or abstain, and fill that tool's arguments. AISA-ArabicFC (Nacar et al., 2026a) poses this problem in Arabic across five dialect families (MSA, Gulf, Egyptian, Levantine, and Maghrebi) and eight service domains. Arabic adds representation load to the argument-filling half, and not where one might expect: the official normaliser already absorbs Arabic-Indic digits and hamza and alef spelling, so those cost nothing under this scorer. In our residual audit, the recurring mismatches are that list items take a comma or the clitic waw, which attaches to the following word; the definite article and generic head nouns are optional on entity values; and values may be written in Arabic or English. We treat this as systematic human label variation rather than random noise (Plank, 2022).

Function accuracy (FnAcc) scores the predicted tool over all rows, so hallucinated and missed calls are both penalised; argument exact-match (ArgEM) compares the whole argument dictionary on positive rows under a published normaliser. Track A ranks by $0.40 \cdot \text{FnAcc} + 0.60 \cdot \text{ArgEM}$, Track B reweights to $0.30 \cdot \text{FnAcc} + 0.50 \cdot \text{ArgEM} + 0.20 \cdot \text{ThinkRate}$, the fraction of rows carrying a reasoning trace, and Track C macro-averages per-dialect Overall A. Our strategy was deliberately simple: one model, no ensembling, no external data, and a few post-processing rules validated on the provided splits.

The dominant gap in this tested setup is argument convention rather than tool selection: under the identical prompt the base model already picks the right tool on 0.9853 of dev rows but matches only 0.6940 of argument dictionaries, and fine-tuning moves arguments while leaving tool choice essentially unchanged. Post-processing adds 0.2 points, and no rule we tested transfers from train to dev. We also report two distinct operational failures. During pre-submission evaluation, our chat endpoint appended a closed, empty reasoning channel; the fine-tune met an unseen prefix, skipped the trace it was trained to emit, and lost 3.8 ArgEM with no error raised. Separately, although we developed the 12B fine-tune before the 31B prompt-only pipeline, we inadvertently omitted it from the blind-test submission.

Two systems appear here and only one was scored. Table 1 and all official numbers are from the submitted prompt-only `gemma-4-31B-it`; Tables 2 and 5 and the residual analysis use a 12B

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1621–1634
October 28-29, 2026 ©2026 Association for Computational Linguistics

fine-tune developed first but omitted from blind-test submission, **except for the convention-card row in Table 5, which evaluates the configured 31B pipeline**. The 12B train, dev, and OOD measurements are not official test results. Re-running the submitted pipeline reproduces 983/1,125 scored calls; its prompt was not version-controlled at submission time and the remote serving state is unrecoverable. Appendix A documents what can and cannot be reconstructed.

The 12B LoRA is publicly available.

2 Background

Task setup. Each instance gives an Arabic query and four candidate tools with JSON schemas; the system outputs a tool name and argument dictionary, or `none`. The Levantine query بدي فندق بالدوحة من ١٨ ل ٢٢ نوفمبر (*biddi funduq bi-l-Dawha min 18 la-22 nufimbir*, 'I want a hotel in Doha from 18 to 22 November') maps to `search_hotels` with `check_in` = 18 نوفمبر and `check_out` = 22 نوفمبر ('18 November', '22 November'): the month is written once and distributed to both endpoints. The digit conversion is free; the month distribution is not.

Data and scoring. The data covers 27 tools over 8 domains, with 10,550 train, 545 dev and 1,125 blind test rows; seven catalog tools are never gold in the released splits and act as distractors. We infer, rather than directly observe, 128 negatives among the 1,125 test rows (11.4%) and therefore 997 positives, the ArgEM denominator; Appendix A reconstructs these counts from the published per-team scores. Negatives are 8.3% of dev and 0.5% of train. ArgEM is computed by an open normaliser that equates different writings of one answer: Arabic-Indic digits, orthographic variants, number words, set-valued fields and alias tables. We entered Tracks A and B; Track C is automatic.

Track	Weighting	FnAcc	ArgEM	Rank
A	0.40/0.60	0.9893	0.8676	**6/17**
B	0.30/0.50/0.20	0.9893	0.8676	**5/17**
C	macro per-dialect	n/a	n/a	**4/21**

Table 1: Official blind-test results for **the 31B prompt-only system**, the strongest of our three submitted `gemma-4-31B-it` variants ($n = 1{,}125$; leaderboard entry *Thaka*). Overall is 0.9163 on Track A, 0.9306 on Track B with ThinkRate 1.000, and 0.8978 on Track C, the macro-average of per-dialect Overall A. The ThinkRate of 1.000 includes traces reconstructed after decoding on rows that emitted none (Section 3), so it measures the presence of a trace and not its consistency with the call.

Relation to prior work. The task follows the AISA reference architecture (Nacar et al., 2026b) and the data-centric fine-tuning methodology behind its baseline (Nacar et al., 2026c). Prior benchmarks such as Gorilla (Patil et al., 2024) and ToolLLM (Qin et al., 2024) are English, and BFCL (Yan et al., 2024) relaxes argument comparison, matching syntax trees and accepting more than one gold value per field. Our results motivate evaluating the second option in a future task edition, without establishing its effect on leaderboard scores. Multilingual coverage has often been reached by translating English data (Chen et al., 2025). The closest independent precedent is Ersoy et al. (2025), whose Arabic error analysis attributes 50.2% of argument errors to paraphrase variance, a semantically correct value in the wrong surface form. Three mechanisms here are ours too: a structured reasoning template rather than free-form chain of thought (Wei et al., 2022; Dang et al., 2025), an Arabic trace before the call (Nacar et al., 2026c; Wei et al., 2026), and repeated-query weighting, related to hard-pattern oversampling (Hao et al., 2025) but not selected by a controlled difficulty score. Our abstention failure is the one Lin et al. (2024) predict, that tuning for function selection degrades irrelevance detection unless negatives are supplied explicitly (Wu et al., 2026). Prior work also trains granular sub-tasks jointly (Abdelaziz et al., 2024), which we do not. Our contribution is a quantitative account of how far annotation convention limits ArgEM under this evaluation protocol, together with the negative results that support it.

3 System Overview

The submitted system. Every leaderboard number here comes from a prompt-only pipeline over `google/gemma-4-31B-it` (Gemma Team, 2026), weights unchanged: greedy decoding under the convention-stating prompt described below, up to four exemplars retrieved per query from the public gold (train and dev for the test run; train alone for every dev number, so no dev query retrieves itself), one per candidate tool and only where the query overlaps it, so the mean is 3.2 and not every query gets four, then eight deter-

ministic post-processing stages, defined in Appendix A: six convention rules, `rare_key_drop`, and `value_canon`. One named rule is score-neutral; the candidate guard used by the fine-tune was not in this pipeline, although it would change no submitted call. Track B traces were completed after decoding: 47 of the 138 abstentions emitted none and received a fixed Arabic sentence, which is why ThinkRate is 1.000 rather than the 0.9582 the decode alone gives (Appendix A).

We entered three variants of this 31B prompt-only pipeline. This paper describes the 31B prompt-only system, the strongest official variant. The earlier 12B fine-tune was inadvertently omitted; the serving defect described in Limitations independently depressed its pre-submission dev measurement.

The fine-tune analysed here. Before developing the submitted 31B pipeline, we fine-tuned `google/gemma-4-12B-it` (Gemma Team, 2026). We did not fine-tune the 31B, which we could reach only as a remote inference endpoint, with LoRA (Hu et al., 2022) over a bfloat16 rather than a quantised base (Dettmers et al., 2023), taking loss on the response tokens only; the training stack is Unsloth (Han et al., 2023) over TRL (von Werra et al., 2020). The system prompt is written in English, carries Arabic terminology and worked examples, and states the annotation conventions explicitly, including schema corrections found by auditing gold (for example `calculate_zakat.type` takes `cash`/`trade`/`shares` rather than the schema's wording). The model emits an Arabic `<think>` trace then a JSON call, or `<no_tool_call>`.

Five deterministic steps follow the fine-tune's decode. (1) Candidate guard: a predicted tool absent from the row's candidate list, or naming one of the seven never-gold distractors, cannot score, so the row abstains. It was worth +0.4 points of FnAcc on the fine-tune's original decode and leaves the corrected decode at 0.9945. (2) Three convention rules, each gated net-positive on train and dev at submission time on the decode our serving defect had corrupted (Limitations). `natural_date` rewrites ISO dates into the natural `<day> <month>` form gold usually uses and strips zero-padded days (no gold date takes the natural form with one, though 84 gold dates are full ISO, and the comparator distinguishes the forms). `customs_origin_not_target` drops `destination_country` when the query marks that country as origin, and `drop_ungrounded_ids` drops identifiers absent from the query. Three further steps drop keys the scorer ignores per tool, fix numerals, currency, punctuation and argument types, and attach the Arabic think text of the model, reconstructed from the final call on rows producing none, which gives ThinkRate 1.000.

After the test phase closed we derived five further rules (dev ArgEM 0.8900 to 0.9020 on the corrected decode). They are part of neither system as evaluated and are reported separately throughout.

4 Experimental Setup

We trained on the provided train split only. The mixture has 12,306 rows over 10,090 unique queries, including 2,216 repeated-query copies inherited from earlier mixture construction rather than a controlled difficulty sampler; the full distribution is in Appendix A. Only 11 copies are negative, so the negative rate remains 0.50%. A row survives only if its rendered query occurs in the provided splits, and the builder aborts if any kept query is test-only, so the guarantee rests on raw query text rather than row identifiers. Every row maps into official train. Test gold was never released. Appendix A gives hyperparameters, software versions and the reproduction procedure. We use the organizers' published scorer unmodified.

Additional resources (post-hoc only). Beyond the provided splits we use one external corpus (Haroon, 2024), strictly for post-competition analysis (Section 5.2). It was not used for training, model selection, rule gating, or any submitted prediction, and not to reconstruct gold for test items; queries overlapping the blind test (1,393 rows) or the provided splits (12,262) were removed before sampling.

5 Results

Table 1 gives the official blind-test results.

Which system these numbers describe. They come from the prompt-only `gemma-4-31B-it` pipeline of Section 3. The fine-tune analysed below was developed earlier but inadvertently omitted from the blind-test submission; it therefore has no official test score.

Leaderboard context. The Track A gap to first is 0.0116 (9 FnAcc and 14 ArgEM rows). Track C

is much less stable: its adjudicated labels contain 34 Maghrebi and 660 MSA rows, so one Maghrebi *FnAcc* error has 19.4 times the weight of one MSA FnAcc error. ArgEM leverage additionally depends on the per-dialect positive counts (Appendix B). Ranking is context here, not evidence for the argument-convention claim.

5.1 The residual on seen and held-out public data

The finished 12B system reaches FnAcc 0.9999 and ArgEM 0.9164 on all 10,550 training rows, versus 0.8880 ArgEM on held-out dev. This does not prove the target unlearnable: only 34 of 878 failures have contradictory duplicate-query gold, so most could be memorised by a larger fit. Under our tested configurations, improvements on training do not transfer to dev; Appendix C quantifies this system-specific result.

Is the residual reachable? Sampling 20 completions per dev positive at temperature 1.0, an answer equivalent to gold under the official `args_match` comparator appears in the pool for 34 of the 56 failing dev rows (60.7%, 95% Wilson CI $[0.48, 0.72]$), and an oracle over it scores 0.9560 on dev and 0.9680 on train. This is scorer equivalence, not byte-identical string equality.

No selector we tested exploits this pool: 289/500 rows have twenty identical samples, while a selector on the rest must win 34 failures without losing 162 greedy holds. Oracle@k can also exploit conflicting gold for identical queries, so we treat it as a diagnostic upper bound rather than an attainable selector result (Appendix C).

5.2 Ablation

Fine-tuning moves arguments far more than tool selection. The three shipped rules add one dev row once correctly served ($0.8880 \rightarrow 0.8900$); the five post-deadline rules add six ($0.8900 \rightarrow 0.9020$) and were never submitted. The fine-tune transfers to the independent annotation ($+14.5$ ArgEM, $+2.1$ FnAcc), whereas convention rules lose 2.0 points there; this is agreement, not correctness (Appendix D). Dev-gated rules also reverse on train: the five later rules fix/break 6/0 dev rows ($p = .031$) but 3/13 train rows ($p = .021$); the three shipped rules yield $+0.20$ dev and -0.04 train.

	dev ($n = 545$)			OOD
Stage	FnAcc	ArgEM	Ov.A	agreement
31B, prompt only‡	0.9945	0.8620	0.9150	—
Base, prompt only	0.9853	0.6940	0.8105	0.5804
Fine-tuned, raw	0.9945	0.8880	**0.9306**	0.7250
+ guard	0.9945	0.8880	0.9306	0.7250
+ conv. rules	0.9945	0.8900	0.9318	0.7054
+ post-dl.†	0.9945	0.9020	0.9390	0.7054

Table 2: Ablation of the **`gemma-4-12B-it` fine-tune**; ‡ is the 31B on dev with train-only retrieval. OOD is argument agreement on 571 independently annotated rows, not accuracy. † is post-deadline; Ov.A $= 0.40$ FnAcc$+0.60$ ArgEM. One run; sub-point effects are not separated from run variance.

5.3 Error analysis (dev, 500 positives)

After the later rules, 49/500 dev positives fail: 11 key-set errors and 38 value mismatches. Gold uses both forms in the largest classes: `get_weather.days` is valued on 535 rows and null on 114, while `order_food.items` is comma-joined on 344 and waw-joined on 126 (125 under a stricter token-initial regex). The comparator splits an attached waw only before a definite noun, so the same separator can be free or fatal. The organisers confirmed this asymmetry and corrected a future release (Appendix C). Our public-split measurements use the public scorer; the official scoring revision is unconfirmed. Also, 30 byte-identical (tool, query) groups carry comparator-distinct gold (76 rows); Appendix C shows that this self-conflict alone bounds little.

Table 5 inventories interventions against this residual. Its positive held-out entries were not significant under the reported paired tests, so smaller gains remain unresolved. A permitted, post-deadline synthetic-negative diagnostic, never submitted, improves train ArgEM by 1.20 points ($p = 6.0\times10^{-12}$) but moves dev by -0.40 ($p = .81$).

6 Conclusion

Positive held-out entries were not statistically significant under the reported paired tests; smaller gains remain unresolved. No tested selector exploited gold found in 20 samples, and multi-reference scoring remains untested. These are setting-specific results; the unsubmitted 12B has no blind-test score.

Limitations

Our conclusions about the limit are drawn from the train and dev splits, because blind-test gold was never released. What we can say about test rests on the published leaderboard aggregates, and our inference of the test negative count is a reconstruction from those aggregates rather than an observation.

Our dialect-level estimates are uncertain. We built no dialect-specific components, several per-dialect dev cells are small, and the participant-facing test file retains labels older than the final adjudicated labels. The organizers confirm that the official values reproduce under the corrected labels, but those row-level labels were not yet available to us; our per-dialect dev numbers are therefore diagnostic rather than predictive.

The 12B fine-tune was developed before the 31B prompt-only pipeline, but we inadvertently omitted it from the blind-test submission. This was an operational submission error, not a dev-based choice between the two systems. A separate serving defect, identified only after the deadline, affected its pre-submission measurement: the chat template appended a closed, empty reasoning channel, which suppressed the trace the fine-tune was trained to emit and depressed its score without raising an error. Neither issue changes the submitted 31B predictions or their official scores. Measurements involving the 12B in this paper use the corrected decode and are flagged where the defective and corrected decodes differ.

The external annotation we cross-check against is an independent labelling of overlapping queries, not a reference for the blind split, so it bounds annotation distance rather than correctness.

We did not implement the proposed field-level multi-reference comparator or rescore other teams. Competitor prediction files and adjudicated row-level test labels were unavailable, so we cannot establish its effect on Track A ordering; our quantified 0.40-point bound covers only byte-identical repeated queries on dev.

We evaluated a single fine-tuning run rather than multiple seeds, so run-to-run variance is not separated from the ablation effects we report, and rules gated on dev are not expected to transfer. ThinkRate measures the presence of a reasoning trace rather than its consistency with the emitted call, and some of our traces were reconstructed after the fact, so that score is not evidence of reasoning-action consistency.

Acknowledgments

We thank Tuwaiq Academy and the AISA-ArabicFC organizers for providing the dataset, the baseline model, and the evaluation framework for this shared task. Releasing the scorer openly is what made the analysis in this paper possible.

References

Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Sadhana Kumaravel, Matthew Stallone, Rameswar Panda, Yara Rizk, GP Bhargav, Maxwell Crouse, Chulaka Gunasekara, Shajith Ikbal, Sachin Joshi, Hima Karanam, Vineet Kumar, Asim Munawar, Sumit Neelam, Dinesh Raghu, Udit Sharma, Adriana Meza Soria, Dheeraj Sreedhar, Praveen Venkateswaran, Merve Unuvar, David Cox, Salim Roukos, Luis Lastras, and Pavan Kapanipathi. 2024. Granite-function calling model: Introducing function calling abilities via multi-task learning of granular tasks. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1131–1139. Association for Computational Linguistics.

Yi-Chang Chen, Po-Chun Hsu, Chan-Jan Hsu, and Dashan Shiu. 2025. Enhancing function-calling capabilities in LLMs: Strategies for prompt formats, data integration, and multilingual translation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*, pages 99–111. Association for Computational Linguistics.

Hy Dang, Tianyi Liu, Zhuofeng Wu, Jingfeng Yang, Haoming Jiang, Tao Yang, Pei Chen, Zhengyang Wang, Helen Wang, Huasheng Li, Bing Yin, and Meng Jiang. 2025. Improving large language models function calling and interpretability via guided-structured templates. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24426–24442. Association for Computational Linguistics.

Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 2022. 8-bit optimizers via block-wise quantization. In *International Conference on Learning Representations (ICLR)*.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*.

Asım Ersoy, Enes Altinisik, Husrev Taha Sencar, and Kareem Darwish. 2025. Tool calling for Arabic LLMs: Data strategies and instruction tuning. In *Proceedings of The Third Arabic Natural Language Processing Conference*, pages 347–358. Association for Computational Linguistics.

Gemma Team. 2026. Gemma 4 technical report. https://arxiv.org/abs/2607.02770. *Preprint*, arXiv:2607.02770. Models: https://huggingface.co/google/gemma-4-31B-it (submitted system) and https://huggingface.co/google/gemma-4-12B-it (fine-tuned system).

Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. 2022. Accelerate: Training and inference at scale made simple, efficient and adaptable. https://github.com/huggingface/accelerate.

Daniel Han, Michael Han, and Unsloth team. 2023. Unsloth. https://github.com/unslothai/unsloth.

Bingguang Hao, ZengZhuang Xu, Maolin Wang, Yuntao Wen, Yicheng Chen, Cunyin Peng, Long Chen, Dong Wang, Xiangyu Zhao, Jinjie Gu, Chenyi Zhuang, and Ji Zhang. 2025. BalanceSFT: Improving LLM function calling with balanced training signals and data hardness. *arXiv preprint arXiv:2505.20192*.

Hesham Haroon. 2024. Arabic function calling dataset. https://huggingface.co/datasets/HeshamHaroon/Arabic_Function_Calling.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

John D. Hunter. 2007. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Neel Jain, Ping-yeh Chiang, Yuxin Wen, John Kirchenbauer, Hong-Min Chu, Gowthami Somepalli, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. 2024. NEFTune: Noisy embeddings improve instruction finetuning. In *International Conference on Learning Representations (ICLR)*.

Josef Jon and Ondřej Bojar. 2024. GAATME: A genetic algorithm for adversarial translation metrics evaluation. In *Proceedings of LREC-COLING*, pages 7562–7569.

Damjan Kalajdzievski. 2023. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Teven Le Scao, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, Joe Davison, Mario Šaško, Gunjan Chhablani, Bhavitvya Malik, Simon Brandeis, Victor Sanh, Canwen Xu, Nicolas Patry, Angelina McMillan-Major, and Thomas Wolf. 2021. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184.

Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, Xiaoyun Mo, Jiamu Zhou, Cheng Cheng, Yin Zhao, Jun Wang, and Weinan Zhang. 2024. Hammer: Robust function-calling for on-device language models via function masking. *arXiv preprint arXiv:2410.04587*.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. 2022. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The first shared task on Arabic function calling for agentic AI systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, and Mohammed Alkhalifa. 2026b. AISA: A unified architecture for agentic AI systems. https://doi.org/10.5281/zenodo.18161880.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, Demah Alhoshan, Rana Alsayyari, Hamed Alruwaili, Albaraa Jaafar, Khaled Alusmani, Abdulaziz Alsohimy, Munirah Alsubaie, Shahd Aldukhayil, Arwa Alali, Yazeed BinShihah, Razan Alsulaymi, Nourah Alhumaid, Razan Abdulsalam, Reem Alamoudi, and Mohammed Alkhalifa. 2026c. From language to action in Arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems 37*.

Barbara Plank. 2022. The "problem" of human label variation: On ground truth in data, modeling and evaluation. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 10671–10682.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations*.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. 2020. TRL: Transformers reinforcement learning. `https://github.com/huggingface/trl`.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Lei Wei, Xiao Peng, Jinpeng Ou, and Bin Wang. 2026. Think-augmented function calling: Improving LLM parameter accuracy through embedded reasoning. *arXiv preprint arXiv:2601.18282*.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

Qinyuan Wu, Soumi Das, Mahsa Amani, Arijit Nag, Seungeon Lee, Krishna Gummadi, Abhilasha Ravichander, and Muhammad Bilal Zafar. 2026. To call or not to call: A framework to assess and optimize LLM tool calling. *arXiv preprint arXiv:2605.00737*.

Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Ion Stoica, Joseph E. Gonzalez, Tianjun Zhang, and Shishir G. Patil. 2024. Berkeley function calling leaderboard. `https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html`.

A Reproducibility Inventory

Table 3 states, for each system in the paper, what data it saw, how it was adapted and how it was decoded.

Licensing. The LoRA adapter is released under the Apache License 2.0. It is a derivative of `gemma-4-12B-it` and is therefore additionally governed by the Gemma Terms of Use; the provided splits are released by the organisers under Apache 2.0.

Repeated-query weighting. The 12,306-row training mixture contains 2,216 copies beyond the first occurrence of a query. Of 1,877 repeated queries, 1,756 occur twice and 121 at least three times. The largest extra-copy concentrations are `get_qibla_direction` (331), `get_weather` (190), and `get_air_quality` (162). This is an inherited weighting, not a controlled hard-example sampler.

Blind-test denominator. Test gold was not released, so we infer the ArgEM denominator from the published scores. Requiring one integer denominator to reproduce all 17 Track A ArgEM values uniquely gives 997 in the range 900–1,125; FnAcc uniquely gives 1,125. We therefore infer 997 positives and 128 negatives rather than treating those counts as directly observed.

Hyperparameters. LoRA rank 64, α 128, dropout 0.05, rsLoRA scaling (Kalajdzievski, 2023), applied to all linear layers of the language tower (attention and MLP; vision layers frozen), giving 262M trainable parameters, 2.15% of the 12.2B total. Three epochs, 1,155 optimizer steps (385 per epoch), per-device batch 1 with gradient accumulation 16 over two GPUs for an effective batch of 32, peak learning rate 5×10^{-5}, cosine schedule with 10% warmup, weight decay 0.01, gradient clipping at 1.0, 8-bit AdamW (Dettmers et al., 2022), NEFTune (Jain et al., 2024) disabled, seed 42, bfloat16, maximum sequence length 4096 at training (the longest mixture row is 4,005 tokens, so no row is truncated) and 6144 at inference. Loss at the final logged step is 0.0035; the mean over the whole run, as the trainer reports it, is 0.2245. Training took 37.5 hours on 2×NVIDIA RTX A6000 (48 GB).

Inference. vLLM 0.27.1 (Kwon et al., 2023), `--max-model-len 6144`, `--gpu-memory-utilization 0.80`, greedy decoding, seed 13, `max_tokens` 768. Output is byte-identical across reruns on fixed hardware and configuration. Decode through `/v1/completions` with the prompt rendered exactly as in training, or set

System	data use	training or retrieval	inference and post-processing
31B, submitted	provided train and dev; no test, no external data	`gemma-4-31B-it`, weights unchanged; up to 4 exemplars per query, one per candidate tool, from train+dev for the test run and train alone for dev measurement; on test the counts are 1/2/3/4 on 17/174/503/431 rows (mean 3.20, four on 38.3%)	greedy, temperature 0; no candidate guard; six convention rules (`procedure_strip`, `medication_strip`, `customs_currency`, `quran_search_type`, `zakat_type_canon`, `termination_canon`), then `rare_key_drop` and `value_canon`; the later trace-filling script was not recovered
12B, developed first; unsubmitted	provided train only; mixture of 12,306 rows over 10,090 unique queries, 0 dev-only and 0 test-only	`gemma-4-12B-it` LoRA rank 64, α 128, 3 epochs, 1,155 steps, seed 42, bf16, 37.5 h on 2×A6000	vLLM 0.27.1, greedy, seed 13, `max_tokens` 768; decode in the training format; candidate guard; three convention rules

Table 3: The two systems and their status. The 12B fine-tune was developed first but inadvertently omitted from the blind-test submission; only the 31B has an official test score. The 12B carries the train, dev, and OOD analyses.

`enable_thinking` to true in the request's `chat_template_kwargs`: without one of the two, Gemma-4's chat template appends a closed, empty reasoning channel, the model skips the trace it was trained to emit, and ArgEM falls by 3.8 points with no error raised. The defect affects fine-tuned checkpoints only. Our officially scored file was produced by the prompt-only pipeline, which never had a training format to mismatch, so the leaderboard figures are unaffected by it; what the defect cost was 3.8 points in the 12B's pre-submission dev measurement, not any part of the score of the 31B system we submitted.

Gemma-4's native tool protocol. We tested whether our bespoke format costs anything against the protocol Google trained in, serving the base model with `--tool-call-parser gemma4`, `--reasoning-parser gemma4` and `--enable-auto-tool-choice` so that calls arrive already parsed. On dev it is worse on both axes: FnAcc 0.8330 and ArgEM 0.5400 with thinking disabled, 0.7486 and 0.4760 with it enabled, against FnAcc 0.9853 and ArgEM 0.6940 for the same base model under our prompt and JSON format. The comparison is not like-for-like, since both the input rendering and the output grammar differ. It shows only that this native-protocol configuration is not a cheaper route to our reported performance; it does not isolate input format from output grammar or prove that formatting cannot contribute to the argument gap.

train ($n = 10{,}550$)	FnAcc	ArgEM
31B, prompt-only	0.9930	0.8132
12B, fine-tuned	0.9999	**0.9164**

Table 4: Both systems scored on the full training split.

31B against 12B, measured. Only the 31B was scored on blind test, but the main-paper dev comparison and every other split we can measure put the 12B fine-tune ahead of the 31B prompt-only pipeline:
The 31B dev figure uses a train-only retrieval pool; with dev in the pool it reaches 0.8640. On the 1,000 test queries the external annotation covers — a different sample from the 571 dialect-balanced OOD rows in Table 2 — the 31B agrees with that annotation on 0.9860 of tool names and 0.6130 of argument dictionaries and the 12B on 0.9940 and 0.6300, so the 12B had the higher agreement with that annotation. This is not an official test score and does not establish a test advantage. The 12B advantage is 2.6 points of dev ArgEM, 10.3 of train, and 1.7 of external argument agreement, against a model 2.6 times its size. The 12B was inadvertently omitted; only the 31B was submitted.

Abstention, and what fine-tuning changed. Dev FnAcc is 0.9853 for the base model and 0.9945 for the fine-tune, but the errors behind those two numbers are disjoint in kind. All 8 of the base model's are missed calls on positive rows and none are false calls; all 3 of the fine-tune's are false calls on negative rows and none are missed. Fine-tuning did not leave tool selection untouched: it moved the failure from under-calling to over-calling, which is what Lin et al. (2024) predict when a model is tuned for selection on a mixture carrying almost no negatives (0.50% of train, against 8.3% of dev). We did not pursue a negatives-augmented fine-tune before the submission deadline. The post-deadline diagnostic in Appendix C tests that lever; on the blind test the FnAcc component of our gap to first place is 9 rows.

Rules on the scored system. The submitted pipeline applied six convention rules of its own, recorded in the producer log:

`procedure_strip`	`quran_search_type`
`medication_strip`	`zakat_type_canon`
`customs_currency`	`termination_canon`

They are disjoint from the three the fine-tune ships, so Table 2 does not describe the scored file's post-processing. The two `strip` rules remove generic procedure or medication heads; `customs_currency` removes customs currency; `quran_search_type` infers a search mode but is score-neutral; `zakat_type_canon` and `termination_canon` map Arabic labels to the gold enums. Then `rare_key_drop` removes scored keys with train emit-rate below 0.02, and `value_canon` applies tool- and field-specific currency, article, head-noun, and procedure-prefix canonicalisation. Our retained implementation records the executable definitions and their order.

Serving the scored system. The prompt-only pipeline that produced the submitted file ran against `gemma-4-31B-it` served in Docker (`vllm/vllm-openai:gemma4`) with tensor parallelism over four GPUs, `--gpu-memory-utilization 0.95` and `--trust-remote-code`. The weights were bind-mounted from local disk to `/gemma-4-31b`, so vLLM advertised that path as the model id; this is the string our logs record, and it is why we cannot attest a Hugging Face revision for the run. `--max-model-len` was not set, so the model default of 32,768 applied. The longest prompt the pipeline builds is 7,081 tokens, and with `max_tokens` 768 the largest request is 7,849, so an 8,192-token window would also have sufficed. The server was additionally started with Gemma-4's native tool-call and reasoning parsers enabled; the pipeline does not use them, parsing the model's JSON output itself. Decoding was greedy, temperature 0, one sample per row, with the anchor date fixed to 2023-10-15.

Reproduction. We ran separate reproduction checks for the two systems. The fine-tune's corrected development prediction file rebuilds byte-exactly from its archived raw outputs. The prompt-only pipeline does not: `run_test.sh` writes `submission_test.jsonl`, whose tool name and argument dictionary match the scored test-split file `predictions/submission_test_v1.jsonl` on 983 of 1,125 rows (87.4%). As a same-era control on the released dev split, we compare `predictions/dev_control_rerun.jsonl` with `predictions/dev_trainonly.jsonl`. Their scored call fields match on 541/545 rows (99.3%) with no function-name differences; byte-identical JSONL rows, including the trace text, number 435/545. The remaining reproduction gap cannot be uniquely attributed to generation or to the unpreserved finalization step described below; we verified that all documented post-processing still runs. The prompt file was not under version control at submission time, but the producer log records a system prompt of 15,677 characters and the recovered file has exactly that length; this evidence supports, but cannot prove byte-for-byte, that its content is unchanged. What we cannot restore is the serving state. The endpoint was remote, served the weights from a local path rather than a released revision, and recycled repeatedly during the window in which we tried to measure it, so we cannot attest a model revision for the original run. We report the scored file itself as the artifact of record.

One further gap, found while preparing this version. The submitted file was written in two steps: the producer script on 28 July, then a finalize step on 29 July that filled the missing Track B traces described in Section 3. That second script was never committed and we could not recover it; its input was also deleted. Its only effect we can still observe is the trace filling, which we can reconstruct exactly because both canned sentences survive in the prediction files, but because its input is gone we can-

not rule out that it also touched tools or arguments. Our reproduction implementation regenerates the eight documented post-processing stages and not that step. On the dev split, `predictions/dev_trainonly.jsonl` therefore reports ThinkRate 0.9633; this is not a like-for-like comparison with the 1.000 reported by the finalized test-split submission.

Software. Training ran on Unsloth 2026.6.9 (Han et al., 2023) over TRL 0.24.0 (von Werra et al., 2020), Transformers 5.10.2 (Wolf et al., 2020), PEFT 0.19.1 (Mangrulkar et al., 2022), Accelerate 1.14.0 (Gugger et al., 2022), bitsandbytes 0.49.2 (Dettmers et al., 2022) (8-bit optimizer), Datasets 4.3.0 (Lhoest et al., 2021) and PyTorch 2.10.0 (Paszke et al., 2019). Serving used vLLM 0.27.1 (Kwon et al., 2023); Transformers below 5.10.2 fails to load the base architecture. Figures use Matplotlib (Hunter, 2007). The scorer files are the organizers' own, verified by checksum.

B Track C: Adjudicated Labels and Small-Cell Leverage

Track C macro-averages the five per-dialect Overall A values. After our query, the organizers confirmed that the official scorer used a corrected, adjudicated dialect assignment with these *total-instance* counts: MSA 660, Egyptian 175, Gulf 128, Levantine 128, and Maghrebi 34 (sum 1,125). The participant-facing `test.parquet` retained an earlier annotation — its dialect column has 825 MSA and 11 Maghrebi rows — and must not be used to recompute Track C. The organizers state that the published leaderboard reproduces under the adjudicated labels and will release them with the task artifacts.

The distinction between total and positive counts matters. If n_d and p_d are a dialect's total and positive counts, a correction affecting only FnAcc moves Track C by $0.2(0.4/n_d)$, while an ArgEM-only correction moves it by $0.2(0.6/p_d)$. Thus a Maghrebi FnAcc error has exactly $660/34 = 19.4$ times the weight of an MSA FnAcc error, but the corresponding ArgEM ratio cannot be computed without the adjudicated positive counts. For a positive Maghrebi row whose function and arguments both flip from wrong to right, the movement is $0.2(0.4/34 + 0.6/p_{\text{Magh}}) \geq 0.0059$, because $p_{\text{Magh}} \leq 34$.

The top five Track C systems span only 0.0118, and our 0.8978 fourth-place score lies within that band. The small Maghrebi cell can therefore change close ordering with very few joint errors. The program committee independently audited this point, concluded that closely placed ordering is not statistically identified, and plans confidence intervals or a larger cell in a future edition. We do not have adjudicated row-level labels or competitor predictions, so we do not claim a paired re-ranking analysis.

Our public-split measurements reinforce only the uncertainty, not a test-set claim. Maghrebi is last on train (ArgEM 0.8310, $n = 142$) but mid-pack on dev, where $n = 6$ gives a Wilson interval of $[0.44, 0.97]$. Our system's FnAcc spread is 1.33 points on dev and 0.02 on train. These diagnostics do not determine its behaviour under the adjudicated blind-test labels.

family (best member)	Δ ArgEM	p
∼15k post-proc. rules	−0.0020	—
retrieval / kNN transfer	−0.0280	—
26 prompt configurations	+0.0040	—
per-tool convention cards†	+0.0020	1.00
selection over 20 samples	+0.0060	0.25
trained pointwise reranker	+0.0060	0.65
noisy-channel reverse scoring	−0.0520	7×10^{-4}
critique-and-revise (train)*	−0.0288	8×10^{-22}
LLM-judge fusion (3 methods)	−0.0240	0.065
cross-model fusion (16 sys.)	≤ 0	—
learned convention classifier	0.0000	—
gold-vocab snapping	−0.0940	2×10^{-13}

Table 5: Exploratory intervention search against the ArgEM residual, **primarily on the `gemma-4-12B-it` fine-tune**. The rows are heterogeneous in split, subset and search procedure and are an inventory of what we tried, not a controlled ablation. Δ is against greedy on the 500 dev positives (one row = 0.0020); p is exact McNemar over discordant pairs. Positive entries were not statistically significant under the reported paired tests; smaller gains remain unresolved. Dashes mark best-of searches, where one paired test would ignore selection over many configurations. † The convention-card row evaluates the configured `gemma-4-31B-it` pipeline; all other rows use the 12B fine-tune. * On a 4,000-row train sample.

C The Residual: Where It Sits and Why Our Tested Rules Did Not Remove It

We classified every failing positive row on both public splits. A row fails either because the tool name is wrong, because the key set differs from gold (KEY-SET), or because every key matches but at least one value does not (VALUE). The pattern be-

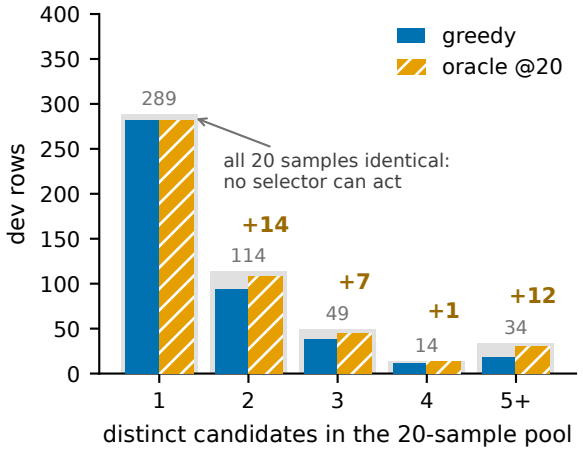


Figure 1: Where the oracle headroom lives for the **`gemma-4-12B-it` fine-tune**. Dev positives ($n = 500$) grouped by how many string-distinct argument dictionaries the 20-sample pool contains; the grey backdrop is the number of rows in each group. Under the scorer's own `args_match` equivalence the first group is larger still, 370 of 500, so the selectability problem below is if anything understated here. On 289 rows all twenty samples agree, so no selector can act, and greedy is already correct on 282 of them. The whole 34-row gap between greedy (0.8880) and oracle (0.9560) sits in the 211 rows with two or more distinct candidates.

low is an instance of human label variation (Plank, 2022): the disagreement is systematic and tied to annotator convention, not random error.

split	failures	VALUE	KEY-SET
train ($n = 10{,}500$)	878	614	264
dev ($n = 500$)	56	44	12

Table 6: Failing positive rows by error type, `gemma-4-12B-it` fine-tune.

Value disagreements are 70% of the train residual and 79% of dev. A correct tool and key set do not alone distinguish convention mismatch from semantic extraction error; in the classes we audited by hand below, what differs is the surface convention rather than the entity referred to. The largest classes are ones where the gold uses both conventions. On train the biggest are `items` (86), `medication_name` (84), `restaurant` (67), `country` (63) and the two `search_hotels` dates (121 combined). The pattern repeats on dev at a twentieth of the scale.

These 70% and 79% figures are the share of our residual assigned to the broad VALUE class, not the share adjudicated as paraphrase variance. They are therefore not directly comparable with the 50.2% paraphrase-variance rate of Ersoy et al. (2025); the two studies support the same qualitative mechanism but use different denominators and taxonomies.

Four kinds account for nearly all of it. A list separator: we write مشويات, مياه معدنية (*mashawiyat, miyah ma'daniyya*, 'grills, mineral water') where gold writes مشويات ومياه معدنية (the same items joined by waw). A definite article, in both directions: we write الباراسيتامول (*al-barasitamul*) for gold باراسيتامول (*barasitamul*, 'paracetamol'), and بروفين (*brufin*) for gold البروفين (*al-brufin*, 'ibuprofen'). A generic head noun, also in both directions: where the query supplies it, gold usually strips it in `restaurant` (kept on 96 of 413 rows) and `medication_name` (32 of 481) but usually keeps it in `procedure` (380 of 531), so the convention inverts between fields. And the language of the value itself, where we answer in Arabic and gold in English, or the reverse.

Self-conflict does not explain this system's residual. A deterministic system maps identical inputs to one output, so self-conflicting gold bounds a perfect system at the largest gold class per identical-query group. Conditioned on the full query that bound is 1.0000 on dev and 0.9971 on train, so self-conflict forbids almost nothing and a ceiling argument resting on it alone would not survive. We can bound what repairing it would buy: of the 76 rows in the 30 self-conflicting groups, only 2 are dev positives, so a multi-reference scorer that accepted every gold observed for a byte-identical (tool, query) pair would move dev ArgEM by at most 0.40 points. That is a bound on this narrow construction only, not on the broader multi-reference gold we recommend, which would also admit alternative surface forms per field. In the features and interventions we tested, we did not identify a convention predictor that transfers to dev. Fitting the majority convention per (tool, field, entity, candidate-set) on train and applying it to dev moves ArgEM from 0.8880 to 0.8800, three rows fixed against seven broken; an entity-agnostic prior reaches 0.8820. The model is already the better-informed predictor: it matches gold exactly on 0.861 of `order_food.restaurant` and 0.848 of `medication_name`, well above the majority-convention rate a surface rule could reach, so such a rule can only overwrite a better decision with a worse one. The fixes-against-breaks table below is

the direct evidence.

Duplicate-query conflicts complicate Oracle@*k*. On the 684 training rows whose query is exactly duplicated elsewhere, gold agrees with its own majority on 0.9561, so no deterministic function of the input can beat that rate on those rows. Separately, Oracle@20 reaches 0.9680 over all training positives. Because the denominators differ and we did not measure Oracle@20 on the 684-row duplicate-query subset, these figures do not establish how much of the all-train oracle gain is unattainable. They show only that, where identical queries carry conflicting gold, a per-row oracle may benefit from choosing different samples for inputs that a deterministic system must map to one answer.

The tested deterministic rules do not remove it. We simulated the obvious deterministic repairs on the train split, applying each to every row of the affected field and counting rows fixed against rows broken. Every one is net-negative:

field	transform	fixes	breaks	net
`items`	comma → waw	42	284	−242
`med_name`	drop ال	30	131	−101
`med_name`	add ال	19	335	−316
`restaurant`	add مطعم	28	319	−291
`restaurant`	drop مطعم	7	52	−45
`category`	drop ال	1	33	−32

Table 7: Deterministic field repairs simulated on train: every candidate is net-negative.

The comparator explains why this field is the worst. It splits list values on a comma, but splits an attached waw only when the word after it begins with the definite article. So مصر والإمارات (*Misr wa-l-Imarat*, 'Egypt and the UAE') matches its comma form while مشويات ومياه معدنية (*mashawiyat wa-miyah ma'daniyya*) does not: the separator is score-free when the following conjunct is definite and fatal when it is not.

Confirmed upstream; not yet in the public scorer revision used here. We reported this asymmetry and the `get_weather.days` inconsistency to the organisers, with two further items: that ArgEM is scored independently of function correctness, so a row abstained on with empty gold arguments still earns argument credit, and that open-vocabulary fields such as `restaurant` and `procedure` admit no Arabic-to-canonical equivalence. The organisers confirmed the comparator asymmetry and replied that it was corrected by broadening the conjunction rule to split an attached waw before any Arabic word, and the second by treating `days` as non-scored for `get_weather`, and that the other two are candidates for a later revision. The public Space itself remained at commit `c43abee` when we re-checked it on 4 September 2026, with `normalize.py` unchanged. Our public-split measurements use that public revision. We have not confirmed which scorer revision produced the official leaderboard results, which we report as returned by the organisers. The confirmed correction is not included in our public-split measurements.

In our comparisons, stating the conventions outright helps only in the following dev-selected setting. We ran the 31B pipeline with per-tool cards that give, for every candidate tool and field, the value form, the emit rate and gold examples mined from train. Against the bare prompt on 228 dev positives the cards are worth $+7.5$ points of ArgEM, 18 rows fixed against 1 broken ($p = 10^{-4}$). Like every configuration in this paper, the cards were selected on development, so this figure carries the same caveat as the rules in Section 5.2: dev-gated gains here did not transfer, and the subset is 228 positives rather than the 500 used elsewhere. Added on top of the shipped configuration, which already retrieves up to four gold exemplars per query, the same cards move 6 rows and break 5 on all 500 dev positives, a net of one row at $p = 1.00$. Four real examples already carry the conventions the cards spell out; adding the cards to the configured 31B pipeline yielded no statistically significant additional dev gain.

Our tested nearest-example transfer does not help either; this result does not bound few-shot prompting in general. Transferring the convention from the most lexically similar same-tool training row scores 0.62 on `items` and 0.75 on `restaurant` against global-majority rates of 0.76 and 0.92, because the nearest neighbour often carries the other convention. In this system's training residual, concentration by domain is stronger than by dialect. By official domain the train ArgEM runs from 0.9942 on Government to 0.7618 on E-commerce, and E-commerce and Healthcare alone hold 467 of the 878 failures; the dialect spread inside a domain is a

third of that. Conditioning on dialect also does not help in this diagnostic. Fitting the majority convention per dialect on train and applying it to dev matches the global majority exactly on `items`, `medication_name` and `restaurant` (no change on any of the 72 rows), because every dialect shares the same majority form. An oracle allowed to pick the best convention per dialect *on dev itself* gains two rows in total. Conditioning on the query does not rescue them. The obvious hypothesis is that gold copies the surface form of the query. It does not: on `order_food.items` the query's separator predicts gold's on 0.283 of rows against 0.672 for always writing a comma, and on `order_food.restaurant` 0.462 against 0.815 for always dropping the head noun. Only `medication_name` favours copying as a feature (0.871 against 0.704), and applied as a rule on train it fixes 20 rows and breaks 41 ($p = 0.010$). The arithmetic explains the failure of these particular transforms. The system is already correct on 75–90% of each field, and gold uses both conventions, so a blanket transform trades a large correct majority for a small incorrect minority. A rule can only win where its conditioning evidence predicts the held-out convention; our tested rules did not.

Three rows of Table 5 deserve comment. Reverse scoring, $\log P(\text{query} \mid \text{args})$, is the only orthogonal signal we found; it fixes 15 rows no other method reaches but breaks 41, and a reranker given it as a feature learns a weight of -0.072, i.e. it ignores it. Gold-vocabulary snapping scores $+38$ rows on train and -47 on dev: the lexicon is built from train gold, so evaluating it on train reads the labels. And optimising directly against the scorer's normaliser, the natural analogue of adversarial metric attacks (Jon and Bojar, 2024), costs 24 rows, because `value_match` is a hard indicator with no partial credit to climb; the same property that makes ArgEM discrete made this tested metric-aware transform ineffective.

Method choice is also constrained by power. With 500 dev positives at ArgEM ≈ 0.89, exact McNemar conditions only on discordant pairs, so a run of six fixes against no breaks already reaches $p = 0.031$ two-sided (0.016 one-sided); no method in Table 5 delivered a run that one-sided. Three interventions looked positive on dev and reversed under replication: a logprob selector at $+1.65$pp on 182 rows fell to $+0.20$pp on 500; `min_p` sampling swung 8 rows across three seeds; and the critic above moved from $+0.80$pp on dev to -2.88pp on 4,000 train rows.

Post-deadline: negative augmentation separates fit from generalisation. Everything in this paragraph is **post-deadline, was never submitted, and is not part of the official system**: the added negatives are synthetic, generated by `gemma-4-31B-it` rather than drawn from the released splits. Synthetic augmentation was allowed by the task; this provenance statement distinguishes the diagnostic from our official entry, which used only the provided data. We report it because it measures directly what the rest of this appendix argues from the failure of other interventions. The mixture is the paper's, plus about 1,019 synthetic no-call rows (13,325 rows, 8.1% negative against 0.50%); the recipe is otherwise unchanged and the comparison is matched at every epoch.

epoch 3	train ArgEM	dev ArgEM	train − dev
no synth. neg.	0.9115	**0.8900**	+2.15
+ negatives	**0.9235**	0.8860	+3.75

Table 8: Negative augmentation: a post-deadline diagnostic using a task-permitted synthetic-negative mixture; never submitted.

On the full 10,550 training rows the augmented run fixes 232 and breaks 106, $+1.20$ points at exact McNemar $p = 6.0\times10^{-12}$. On dev it fixes 8 and breaks 10, -0.40 points at $p = 0.81$. A method that moves training ArgEM about as decisively as this data allows moves held-out ArgEM by nothing and widens the train–dev gap from 2.15 to 3.75 points. Thus this particular augmentation learns additional training conventions without a corresponding dev gain; it does not establish a task-wide ceiling or impossibility result.

The abstention result is smaller than it first looks and we state it as such. Negatives fix 10 FnAcc rows against 0 at epoch 2 ($p = 0.0020$), but by epoch 3 the margin is 5 against 1 ($p = 0.22$) and the original-mixture arm has caught up on its own from 61 negatives. The augmented arm reached fewer FnAcc errors earlier by epoch count, but its final advantage was not statistically significant: epoch 2 with augmentation (1 FnAcc error) already beats epoch 3 without it (6). Two caveats. The baseline arm here is a matched re-run for this experiment, not the fine-tune of Table 2: it reaches 0.9890 FnAcc against that model's 0.9945, so the comparison is internal to this appendix and neither arm should be read against 0.8880. Both arms are scored on raw decodes, and this is one run

per arm, so sub-point differences are not separated from run-to-run variance. Finally, this augmentation changes the training data, not model capacity: it improved training ArgEM without a statistically significant dev improvement, and higher-rank or longer-training experiments remain untested.

D External Annotation Cross-Check

We checked whether a separately released Arabic function-calling corpus (Haroon, 2024) could stand in for the blind test gold. It cannot, and the reason is instructive.

Measured on the 11,000 rows where both resources label the same query, and scored with the official comparator, the two annotations agree perfectly on the function name (11,000/11,000) and on only 0.7056 of argument dictionaries. The disagreement concentrates in a few tools, and splits them cleanly into two groups (Table 9). The tools that disagree are those whose arguments are dates, money and free-text lists, which is where the conventions of these two annotation resources diverge most.

We report this as evidence that argument conventions differ sharply between independent annotations of the same task. We did not use it to estimate a test score: it is an independent annotation of overlapping queries, not a reference for the blind split, and the tools on which it diverges most are exactly the ones that carry our own residual.

tool	agreement
agree ≥ 0.95	
`check_traffic_violations`	0.998
`translate_text`	0.997
`search_quran`	0.996
`get_air_quality`	0.992
`check_iqama_status`	0.991
`get_qibla_direction`	0.991
`convert_currency`	0.985
`check_visa_status`	0.981
`search_medications`	0.951
agree ≤ 0.24	
`search_hotels`	0.240
`transfer_money`	0.017
`calculate_customs`	0.000
`calculate_end_of_service`	0.000
`search_umrah_packages`	0.000

Table 9: Model-independent per-tool argument agreement between the external annotation and AISA gold, over the 11,000 shared rows. Tools not listed fall between these groups; no system predictions enter this table.